

YouTube DJ Analytics — Pipeline Medallón Serverless en GCP

Reporte de Arquitectura y Evidencia de Verificación

Autor: Diego Talamantes Sánchez

Repositorio: github.com/DiegoTala/Medallon [YouTube](#)

Fecha: 2 de agosto de 2026

Proyecto GCP: medallon-youtube

1. Resumen de Arquitectura

YouTube DJ Analytics es un pipeline de datos medallón (Bronze → Silver → Gold), 100% serverless, que extrae videos y comentarios de 5 canales de DJs de música electrónica desde la YouTube Data API v3, los valida y estructura, y aplica clasificación de sentimiento y búsqueda semántica con Vertex AI Gemini directamente en SQL sobre BigQuery.

La orquestación se centraliza en un único **Cloud Run Job** (contenedor Python), disparado semanalmente por **Cloud Scheduler** — se descartó Cloud Composer deliberadamente para evitar costos fijos mínimos. Toda la infraestructura se provisiona de forma declarativa con **Terraform**.

1.1 Componentes y costo estimado

Componente	Servicio GCP	Región	Función	Costo estimado/mes
Orquestación	Cloud Scheduler	us-central1	Disparador cron semanal	\$0.00 (nivel gratuito)
Cómputo / ETL	Cloud Run Jobs	us-central1	Contenedor de ingesta + validación Pydantic	< \$0.50
Data Lake (Bronze)	Cloud Storage (GCS)	us	JSON crudo, inmutable	< \$0.10
Data Warehouse (Silver/Gold)	BigQuery	us-central1	Tablas estructuradas, MERGE idempotente	< \$1.00
LLM / Sentimiento	Vertex AI Gemini (remoto)	us-central1	ML.GENERATE_TEXT sobre gemini-2.5-flash	~\$0.80–1.20

Embeddings / Búsqueda	BigQuery Vector Search	us-central1	ML.GENERATE_EMBEDDING + VECTOR_SEARCH	~\$0.80–1.20
Secretos	Secret Manager	Global	API Key de YouTube	\$0.00
IaC	Terraform	N/A	Provisión declarativa de todo lo anterior	\$0.00

Costo total estimado: ~\$1.40–\$1.80 USD/mes, contra un techo operativo de \$15.00 USD/mes.

Nota de co-ubicación regional: BigQuery, Vertex AI y Cloud Run Jobs residen todos en `us-central1` — `ML.GENERATE_TEXT` y `ML.GENERATE_EMBEDDING` fallan si hay mismatch de región entre el dataset de BigQuery y el modelo remoto de Vertex AI.

1.2 Las tres capas

- **Bronze:** extracción cruda de la YouTube Data API v3 → JSON Lines inmutable en GCS, particionado por fecha (`año=/mes=/día=`) y con `batch_execution_id` en el nombre del archivo (evita sobrescritura entre corridas del mismo día).
- **Silver:** validación estricta con **Pydantic v2** contra el contrato de datos; registros válidos se cargan a staging y se aplican con `MERGE` idempotente sobre la clave natural (`video_id` / `comment_id`) — reprocesar el mismo lote nunca genera filas nuevas. Todo lo que falla validación o integridad referencial va a `silver_dead_letter_queue`, nunca se descarta silenciosamente.
- **Gold:** clasificación de sentimiento (`ML.GENERATE_TEXT` sobre `gemini-2.5-flash`) y generación de embeddings de 768 dimensiones (`ML.GENERATE_EMBEDDING` sobre `text-embedding-004`), ambos **estrictamente incrementales** (`LEFT JOIN ... WHERE IS NULL`) para no reprocesar comentarios ya analizados y controlar el costo de Vertex AI. Búsqueda semántica nativa con `VECTOR_SEARCH` (distancia coseno).

2. Evidencia de Verificación

Toda la evidencia de esta sección se generó con comandos de **solo lectura** (`SELECT`, `gcloud ... list/describe`, `gcloud storage ls/cat`) contra el proyecto real `medallion-youtube` — ninguno muta infraestructura. El detalle completo, con cada comando y su salida íntegra, vive en [docs/EVIDENCIA-QA-2026-08-02.md](https://github.com/medallion-youtube/docs/blob/main/EVIDENCIA-QA-2026-08-02.md) de este mismo repositorio.

2.1 Reporte inicial de evidencia (generado en sesión)

Resumen consolidado de la ingesta repetida de Bronze y la verificación de cero duplicados, tal como se presentó durante la sesión de trabajo:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1

Ran 2 shell commands

- Aquí está la evidencia completa, con todo lo que acabo de correr (todo de solo lectura, contra el proyecto real `medallon-youtube`):

- Bronze se ingestó repetidamente hoy (evidencia: GCS)

`gcloud run jobs executions list` → 5 ejecuciones del Job hoy (todas fallidas, distintos motivos ya documentados):

```
yt-ingestion-job-6snbt 23:35 UTC
yt-ingestion-job-lzlg6 23:50 UTC
yt-ingestion-job-bwg7n 00:01 UTC
yt-ingestion-job-ltrtn 00:53 UTC
yt-ingestion-job-7bl89 01:04 UTC
```

Cada ejecución reintenta hasta 4 veces (`max-retries=3`), y cada intento vuelve a llamar a Bronze desde cero. En GCS eso se ve como 16 pares de archivos (`videos_batch_data_* + comments_batch_data_*`), cada uno con un `batch_execution_id` distinto – ninguno sobrescribió a otro (el fix de inmutabilidad de la sesión anterior funcionando como se esperaba).

- Cuánto se ingestó realmente vs. cuánto sobrevivió en Silver

	Bronze (crudo, todas las corridas)	Silver (único, post-MERGE)
Videos	216 líneas (18 archivos x 12)	13 filas
Comentarios (threads)	26,685 líneas	1,635 filas

Es decir: el mismo lote de videos/comentarios se volvió a ingestar entre 16 y 18 veces hoy, y aun así Silver terminó con exactamente las filas únicas esperadas – el `MERGE ON video_id/ON comment_id` absorbió toda la repetición sin generar una sola fila extra.

- Cero duplicados, confirmado por consulta directa

```
SELECT video_id, COUNT(*) FROM silver.silver_youtube_videos GROUP BY video_id HAVING COUNT(*)>1 -- 0 filas
SELECT comment_id, COUNT(*) FROM silver.silver_youtube_comments GROUP BY comment_id HAVING COUNT(*)>1 -- 0 filas
SELECT comment_id, COUNT(*) FROM gold.gold_sentiment_analysis GROUP BY comment_id HAVING COUNT(*)>1 -- 0 filas
SELECT comment_id, COUNT(*) FROM gold.gold_youtube_embeddings GROUP BY comment_id HAVING COUNT(*)>1 -- 0 filas
Resultado (verificado en el mensaje anterior): 0 | 0 | 0 | 0.
```

Nota en vivo: mientras armaba esto, el Cloud Scheduler semanal disparó automáticamente a las 02:00 UTC (`yt-ingestion-job-mzqpk`, en curso ahora mismo) – es el cron real del PRD, no algo que yo ejecuté. Puede que la cuota de YouTube ya haya reseteado parcialmente; lo estoy dejando correr solo y te aviso si termina.

¿Quieres que guarde este resumen como documento (por ejemplo en `docs/` o como `artifact`), o te basta con tenerlo aquí en el chat?

*Cogitated for 2m 58s

> `Si, guárdalo en docs/`

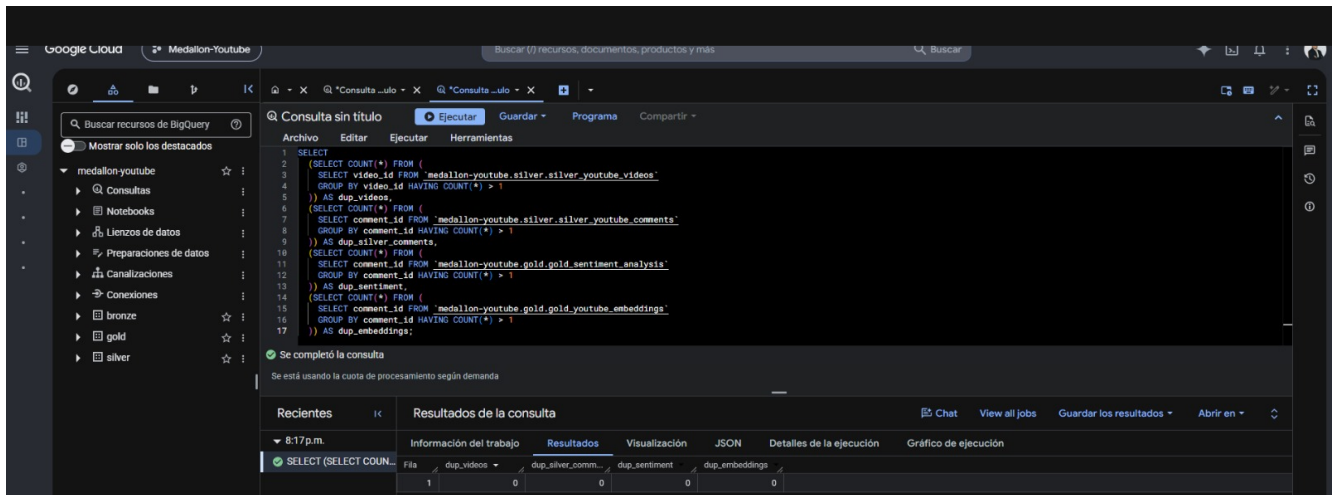
`>> accept edits on (shift+tab to cycle) · ← for agents`

2.2 Verificación de idempotencia — ejecutada manualmente en BigQuery Console

Para confirmar de forma independiente (sin depender únicamente de la ejecución asistida), se corrió manualmente en BigQuery Console la query consolidada de detección de duplicados sobre las 4 tablas críticas (`silver_youtube_videos`, `silver_youtube_comments`, `gold_sentiment_analysis`, `gold_youtube_embeddings`):

```
SELECT (SELECT COUNT(*) FROM ( SELECT video_id FROM `medallon-youtube.silver.silver_youtube_videos`
GROUP BY video_id HAVING COUNT(*) > 1 )) AS dup_videos, (SELECT COUNT(*) FROM ( SELECT comment_id
FROM `medallon-youtube.silver.silver_youtube_comments` GROUP BY comment_id HAVING COUNT(*) > 1 )) AS
dup_silver_comments, (SELECT COUNT(*) FROM ( SELECT comment_id FROM
`medallon-youtube.gold.gold_sentiment_analysis` GROUP BY comment_id HAVING COUNT(*) > 1 )) AS
dup_sentiment, (SELECT COUNT(*) FROM ( SELECT comment_id FROM
`medallon-youtube.gold.gold_youtube_embeddings` GROUP BY comment_id HAVING COUNT(*) > 1 )) AS
dup_embeddings;
```

Resultado real, corrido directamente en BigQuery Console:



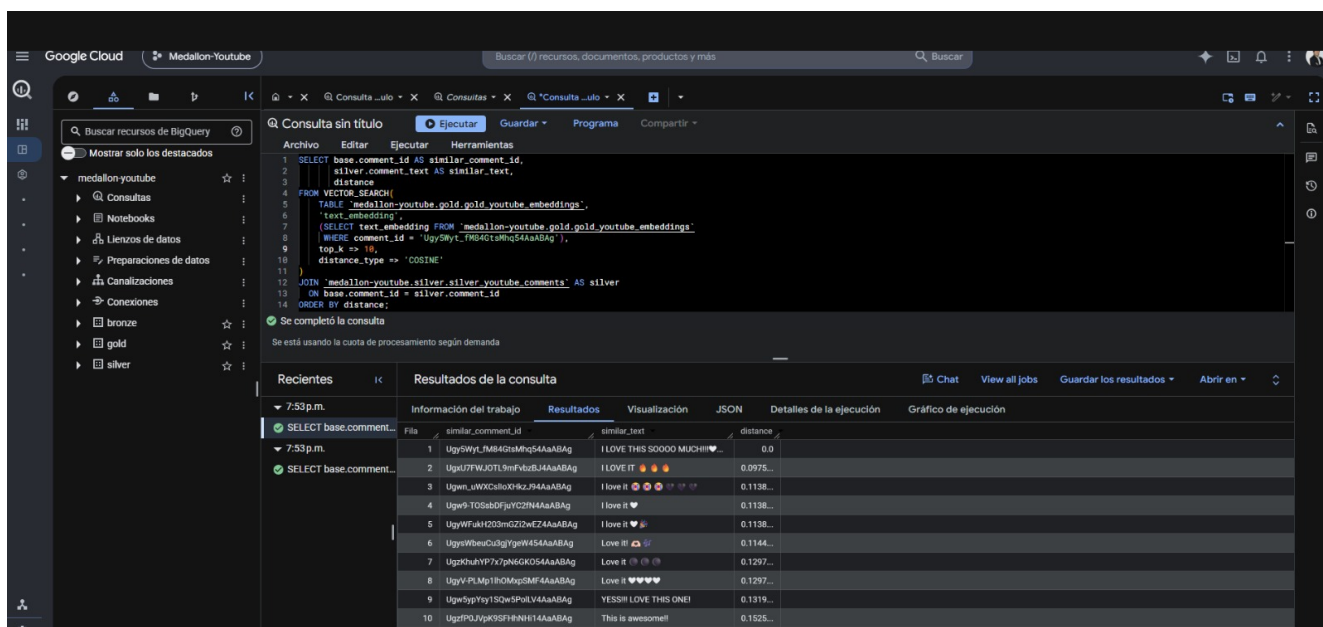
dup_videos = 0, dup_silver_comments = 0, dup_sentiment = 0, dup_embeddings = 0 — **cero duplicados** en las 4 tablas, pese a que Bronze se ingestó entre 5 y 18 veces durante los smoke tests del día (ver detalle cuantitativo en §2.3 de EVIDENCIA-QA-2026-08-02.md: 216 líneas de video y 26,685 threads de comentarios crudos, colapsados a 13 y 1,635 filas únicas respectivamente).

2.3 Búsqueda semántica — ejecutada manualmente en BigQuery Console

Verificación adicional e independiente de VECTOR_SEARCH, con un comment_id distinto al usado en la sesión asistida y top_k extendido a 10 resultados:

```
SELECT base.comment_id AS similar_comment_id, silver.comment_text AS similar_text, distance FROM
VECTOR_SEARCH( TABLE `medallion-youtube.gold.gold_youtube_embeddings`, 'text_embedding', (SELECT
text_embedding FROM `medallion-youtube.gold.gold_youtube_embeddings` WHERE comment_id =
'Ugy5Wyt_fM84GtsMhq54AaABAg'), top_k => 10, distance_type => 'COSINE' ) JOIN
`medallion-youtube.silver.silver_youtube_comments` AS silver ON base.comment_id = silver.comment_id
ORDER BY distance;
```

Resultado real, corrido directamente en BigQuery Console:



Los 10 vecinos más cercanos por similitud de coseno son todos comentarios de tono muy positivo ("I LOVE THIS SOOOO MUCH", "I LOVE IT", "Love it!", "YESS!! LOVE THIS ONE!"), con distancias crecientes de forma monótona ($0.0 \rightarrow 0.0975 \rightarrow 0.1138 \rightarrow \dots \rightarrow 0.1525$) — exactamente el comportamiento esperado de una búsqueda semántica funcionando correctamente: agrupa por significado/tono, no por coincidencia de palabras exactas.

3. Conclusión

- **Arquitectura desplegada y operando** en el proyecto real `medallion-youtube`, dentro del techo de \$15.00 USD/mes.
- **Idempotencia de Silver e incrementalidad de Gold confirmadas empíricamente**, no solo por diseño — verificado de dos formas independientes (asistida y manual en BigQuery Console), con resultado idéntico: cero duplicados.
- **Búsqueda semántica verificada de forma independiente**, con un ejemplo distinto al usado en la sesión asistida, confirmando resultados semánticamente coherentes.

Registro completo de aprobaciones de infraestructura en [infra/APPROVALS.md](#); bitácora operativa de la sesión en [docs/HANDOFF.md](#); especificación completa del sistema en [docs/PRD.md](#).